

Python for Computational Problem Solving Lab

Hackathon - Jackfruit Level Problem solving

Date: 02/01/2025

Team Details: C7 Section

- 1) Guru R Bharadwaj : PES1UG24CS177
- 2) Gaurav Shivkumar : PES1UG24AM104
- 3) Himesh Raghav : PES1UG24CS188
- 4) Jeremy Nischal Samuel : PES1UG24EE021

Code:

```
import csv
import datetime
from tkinter import *
from tkinter import messagebox

BASE_PENALTY_RATE = 1
MAX_PENALTY = 50
GRACE_PERIOD = 2

class Book:
    def __init__(self, book_id, title, author, available_copies):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.available_copies = available_copies

    def to_csv(self):
        return [self.book_id, self.title, self.author, self.available_copies]

class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.borrowed_books = []

    def add_borrowed_book(self, book_id):
```

Python for Computational Problem Solving Lab

```
if len(self.borrowed_books) < 5:
    self.borrowed_books.append(book_id)
    return True
return False

def remove_borrowed_book(self, book_id):
    if book_id in self.borrowed_books:
        self.borrowed_books.remove(book_id)
        return True
    return False

def to_csv(self):
    return [self.student_id, self.name, ','.join(self.borrowed_books)]

def is_eligible_to_borrow(self):
    return len(self.borrowed_books) < 5

class Librarian:
    def __init__(self):
        self.books = []
        self.students = []
        self.load_data()

    def load_data(self):
        # Loading books data
        try:
            with open('books.csv', newline='') as csvfile:
                reader = csv.reader(csvfile)
                for row in reader:
                    if len(row) >= 4: # Ensure there are at least 4 columns
                        book = Book(row[0], row[1], row[2], int(row[3])) # Ensure available_copies is an integer
                        self.books.append(book)
                    else:
                        print(f"Skipping malformed row in books data: {row}")
        except FileNotFoundError:
            print("Books data file not found.")

        # Loading students data
        try:
            with open('students.csv', newline='') as csvfile:
                reader = csv.reader(csvfile)
                for row in reader:
                    if len(row) >= 3: # Ensure there are at least 3 columns
```

Python for Computational Problem Solving Lab

```
student = Student(row[0], row[1])
student.borrowed_books = row[2].split(',') if row[2] else []
self.students.append(student)
else:
    print(f"Skipping malformed row in students data: {row}")
except FileNotFoundError:
    print("Students data file not found.")

def save_data(self):
    with open('books.csv', 'w', newline='') as csvfile:
        writer = csv.writer(csvfile)
        for book in self.books:
            writer.writerow(book.to_csv())

    with open('logs.csv', 'w', newline='') as csvfile:
        writer = csv.writer(csvfile)
        for student in self.students:
            writer.writerow(student.to_csv())

def check_stock(self):
    return [(book.book_id, book.title, book.available_copies) for book in self.books if
book.available_copies > 0]

def search_book(self, query):
    results = [book for book in self.books if query.lower() in book.title.lower() or query.lower() in
book.author.lower()]
    return results

def search_student(self, query):
    results = [student for student in self.students if query.lower() in student.name.lower()]
    return results

def issue_book(self, book_id, student_id):
    book = next((b for b in self.books if b.book_id == book_id), None)
    student = next((s for s in self.students if s.student_id == student_id), None)

    if not book or not student:
        return "Book or Student not found."

    if book.available_copies <= 0:
        return "Book is not available."

    if not student.is_eligible_to_borrow():
```

Python for Computational Problem Solving Lab

```
return "Student has reached the borrowing limit."
```

```
student.add_borrowed_book(book.book_id)
book.available_copies -= 1
self.save_data()
```

```
# Log the issued book only
self.log_issue(book, student)
```

```
return f"Book '{book.title}' issued to {student.name}."
```

```
def return_book(self, book_id, student_id):
    book = next((b for b in self.books if b.book_id == book_id), None)
    student = next((s for s in self.students if s.student_id == student_id), None)
```

```
if not book or not student:
    return "Book or Student not found."
```

```
if not student.remove_borrowed_book(book.book_id):
    return "This book was not borrowed by the student."
```

```
return_date = datetime.datetime.now().date()
due_date = datetime.date.today() - datetime.timedelta(days=GRACE_PERIOD)
late_days = (return_date - due_date).days
```

```
penalty = 0
if late_days > 0:
    penalty = min(late_days * BASE_PENALTY_RATE, MAX_PENALTY)
```

```
book.available_copies += 1
self.save_data()
```

```
return f"Book '{book.title}' returned. Penalty: ${penalty}." if penalty > 0 else f"Book '{book.title}'
returned. No penalty."
```

```
def log_issue(self, book, student):
    # Open the log file in append mode
    with open('logs.csv', 'a', newline='') as logfile:
        writer = csv.writer(logfile)
        issue_date = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        writer.writerow([issue_date, "Issued", book.book_id, book.title, student.student_id,
student.name])
```

Python for Computational Problem Solving Lab

```
class LibraryManagementSystem:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        root.title("Library Management System")
```

```
        self.librarian = Librarian()
```

```
        Label(root, text="Book Search").grid(row=0)
```

```
        self.book_search_entry = Entry(root)
```

```
        self.book_search_entry.grid(row=0, column=1)
```

```
        Button(root, text="Search Book", command=self.search_book).grid(row=0, column=2)
```

```
        Label(root, text="Available Books").grid(row=1)
```

```
        self.book_results_text = Text(root, height=10, width=50)
```

```
        self.book_results_text.grid(row=1, column=1)
```

```
        Label(root, text="Student Search").grid(row=2)
```

```
        self.student_search_entry = Entry(root)
```

```
        self.student_search_entry.grid(row=2, column=1)
```

```
        Button(root, text="Search Student", command=self.search_student).grid(row=2, column=2)
```

```
        Label(root, text="Student Info").grid(row=3)
```

```
        self.student_info_text = Text(root, height=10, width=50)
```

```
        self.student_info_text.grid(row=3, column=1)
```

```
        Label(root, text="Issue Book").grid(row=4)
```

```
        Label(root, text="Book ID").grid(row=4, column=1)
```

```
        Label(root, text="Student ID").grid(row=4, column=2)
```

```
        self.issue_book_entry = Entry(root)
```

```
        self.issue_student_entry = Entry(root)
```

```
        self.issue_book_entry.grid(row=4, column=3)
```

```
        self.issue_student_entry.grid(row=4, column=4)
```

```
        Button(root, text="Issue", command=self.issue_book).grid(row=4, column=5)
```

```
        Label(root, text="Return Book").grid(row=5)
```

Python for Computational Problem Solving Lab

```
Label(root,text="Book ID").grid(row=5,column=1)
```

```
Label(root,text="Student ID").grid(row=5,column=2)
```

```
self.return_book_entry = Entry(root)
```

```
self.return_student_entry = Entry(root)
```

```
self.return_book_entry.grid(row=5,column=3)
```

```
self.return_student_entry.grid(row=5,column=4)
```

```
Button(root,text="Return",command=self.return_book).grid(row=5,column=5)
```

```
def search_book(self):
```

```
    query = self.book_search_entry.get()
```

```
    results = self.librarian.search_book(query)
```

```
    display_text = ""
```

```
    if results:
```

```
        for result in results:
```

```
            display_text += f"ID: {result.book_id}, Title: {result.title}, Available Copies:
```

```
{result.available_copies}\n"
```

```
    else:
```

```
        display_text = "No results found."
```

```
    self.book_results_text.delete(1.0, END)
```

```
    self.book_results_text.insert(END, display_text)
```

```
def search_student(self):
```

```
    query = self.student_search_entry.get()
```

```
    results = self.librarian.search_student(query)
```

```
    display_text = ""
```

```
    if results:
```

```
        for result in results:
```

```
            display_text += f"ID: {result.student_id}, Name: {result.name}, Borrowed Books: {'
```

```
    else:
```

```
        display_text = "No results found."
```

```
    self.student_info_text.delete(1.0, END)
```

```
    self.student_info_text.insert(END, display_text)
```

```
def issue_book(self):
```

```
    book_id = self.issue_book_entry.get()
```

```
    student_id = self.issue_student_entry.get()
```

```
    if not book_id or not student_id:
```

```
        messagebox.showerror("Error", "Please enter both Book ID and Student ID.")
```

Python for Computational Problem Solving Lab

```
return
```

```
result = self.librarian.issue_book(book_id, student_id)
```

```
if "issued" in result:
```

```
    messagebox.showinfo("Issue Book", result)
```

```
else:
```

```
    messagebox.showwarning("Issue Book", result)
```

```
def return_book(self):
```

```
    book_id = self.return_book_entry.get()
```

```
    student_id = self.return_student_entry.get()
```

```
    if not book_id or not student_id:
```

```
        messagebox.showerror("Error", "Please enter both Book ID and Student ID.")
```

```
    return
```

```
result = self.librarian.return_book(book_id, student_id)
```

```
if "returned" in result:
```

```
    messagebox.showinfo("Return Book", result)
```

```
else:
```

```
    messagebox.showwarning("Return Book", result)
```

```
if __name__ == "__main__":
```

```
    root = Tk()
```

```
    app = LibraryManagementSystem(root)
```

```
    root.mainloop()
```

Python for Computational Problem Solving Lab

Output (GUI) Screenshots:

Library Management System

Book Search: Search Book

Available Books: ID: B001, Title: The Great Gatsby, Available Copies: 5

Student Search: Search Student

Student Info: ID: S001, Name: John Doe, Borrowed Books:

Issue Book: Book ID: Student ID: B001 S001 Issue

Return Book: Book ID: Student ID: B001 S001 Return

Library Management System

Book Search: Search Book

Available Books: ID: B001, Title: The Great Gatsby, Available Copies: 5
ID: B002, Title: To Kill a Mockingbird, Available Copies: 2
ID: B003, Title: 1984, Available Copies: 4
ID: B004, Title: Moby-Dick, Available Copies: 2
ID: B005, Title: War and Peace, Available Copies: 0
ID: B006, Title: The Catcher in the Rye, Available Copies: 6

Student Search: Search Student

Student Info: ID: S001, Name: John Doe, Borrowed Books:
ID: S002, Name: Alice Smith, Borrowed Books:
ID: S003, Name: Bob Johnson, Borrowed Books:
ID: S004, Name: Charlie Brown, Borrowed Books:

Issue Book: Book ID: Student ID: Issue

Return Book: Book ID: Student ID: Return

Return Book

Book 'The Great Gatsby' returned. Penalty: \$2.

OK

Python for Computational Problem Solving Lab

Library Management System

Book Search Search Book

Available Books

- ID: B001, Title: The Great Gatsby, Available Copies: 5
- ID: B002, Title: To Kill a Mockingbird, Available Copies: 2
- ID: B003, Title: 1984, Available Copies: 4
- ID: B004, Title: Moby-Dick, Available Copies: 2
- ID: B005, Title: War and Peace, Available Copies: 0
- ID: B006, Title: The Catcher in the Rye, Available Copies: 6

Student Search Search Student

Student Info

- ID: S001, Name: John Doe, Borrowed Books:
- ID: S002, Name: Alice Smith, Borrowed Books:
- ID: S003, Name: Bob Johnson, Borrowed Books:
- ID: S004, Name: Charlie Brown, Borrowed Books:

Issue Book Book ID

Return Book Book ID

Issue Book

Book 'The Great Gatsby' issued to John Doe.

OK